

Apunte de Cátedra

ALGORITMOS DE ORDENAMIENTO

1.- INTRODUCCION

Dado un conjunto de n elementos $a_1, a_2, \dots, a_n$ y una relación de orden total ($\leq$) sobre ellos, el problema de la ordenación consiste en encontrar una permutación de esos elementos ordenada de forma creciente.

Aunque tanto el tipo y tamaño de los elementos como el dispositivo en donde se encuentran almacenados pueden influir en el método que utilicemos para ordenarlos, en este tema vamos a solucionar el caso en que los elementos son números enteros y se encuentran almacenados en un arreglo.

Si bien existen distintos criterios para clasificar a los algoritmos de ordenación, una posibilidad es atendiendo a su eficiencia. De esta forma, en función de la complejidad que presentan en el caso medio, podemos establecer la siguiente clasificación:

- $\Theta(n^2)$: Burbuja, Inserción, Selección, ShakerSort.
- $\Theta(n \log n)$: MergeSort, HeapSort, QuickSort.
- Otros: ShellSort $\Theta(n^{1.25})$, BinSort $\Theta(n)$, RadixSort $\Theta(n)$.

Otra clasificación que podemos hacer según su tipo de ordenación, es la siguiente:

- Ordenaciones por intercambio: Burbuja, ShakerSort, QuickSort.
- Ordenaciones por selección: Selección, HeapSort.
- Ordenaciones por inserción: Inserción, ShellSort.
- Ordenaciones por distribución: BinSort, RadixSort.
- Ordenaciones por intercalación: MergeSort.

A continuación, desarrollaremos todos ellos con detenimiento, prestando especial atención a su complejidad, no sólo en el caso medio sino también en los mejores y peores casos, pues para algunos existen diferencias significativas.

Como hemos mencionado anteriormente, nos centraremos en la ordenación de enteros, muchos de los problemas de ordenación que nos encontramos en la práctica son de ordenación de datos más complejos. Sin embargo este problema puede ser fácilmente reducido al de ordenación de números enteros utilizando las claves o bien índices. Por otro lado, puede que los datos a ordenar excedan la capacidad de memoria del ordenador, y por lo tanto, deban residir en dispositivos externos. Aunque este problema, denominado *ordenación externa*, presenta ciertas dificultades específicas, los métodos utilizados para resolverlo se basan fundamentalmente en los algoritmos que aquí presentamos.

Antes de pasar a desarrollar los principales algoritmos, es necesario aclarar que todos los métodos presentados van incluidos en la siguiente clase:

```
public final class Sorts
{
}
```

2.- ALGORITMOS BASICOS DE ORDENAMIENTO

2.1.- Ordenamiento por Inserción

El método de Inserción realiza $n-1$ iteraciones sobre el arreglo, dejando en la i -ésima etapa ($2 \leq i \leq n$) ordenado el arreglo $\text{vec}[1 \dots i]$. La forma de hacerlo es colocando en cada iteración el elemento $\text{vec}[i]$ en su sitio correcto, aprovechando el hecho de que el arreglo $\text{vec}[1 \dots i-1]$ ya ha sido previamente ordenado. Este método puede ser implementado de forma iterativa como sigue:

Apunte de Cátedra

```
public static void insertionSort(int[] vec)
{
    for(int i=1; i<vec.length; i++)
    {
        int aux=vec[i];
        int j=i-1;
        while(j>=0 && vec[j]>aux)
        {
            vec[j+1]=vec[j];
            j-=1;
        }
        vec[j+1]=aux;
    }
}
```

Para estudiar su complejidad, vamos a estudiar los casos mejor, peor y medio de la llamada al método *insertionSort(vec)*.

- En el mejor caso el bucle interno no se realiza nunca, y por lo tanto:

$$T(n) = \left(\sum_{i=1}^{n-1} (3 + 4 + 4 + 3) \right) + 3 = 14n - 11.$$

- En el peor caso hay que llevar cada elemento hasta su posición final, con lo que el bucle interno se realiza siempre de $i-1$ veces. Así, en este caso:

$$T(n) = \left(\sum_{i=1}^{n-1} \left(3 + 4 + \left(\sum_{j=0}^{i-1} (4 + 5) \right) + 1 + 3 \right) \right) + 3 = \frac{9}{2} n^2 + \frac{13}{2} n - 10.$$

- En el caso medio, supondremos equiprobable la posición de cada elemento dentro del arreglo. Por lo tanto, para cada valor de i , la probabilidad de que el elemento se sitúe en alguna posición k de las i primeras será de $1/i$. El número de veces que se repetirá el bucle while en este caso es $(i-k)$, con lo cual el número medio de operaciones que se realizan en el bucle es:

$$\left(\frac{1}{i} \sum_{k=1}^i 9(i-k) \right) + 4 = \frac{9}{2} i - \frac{1}{2}.$$

Por lo tanto, el tiempo de ejecución en el caso medio es:

$$T(n) = \left(\sum_{i=1}^{n-1} \left(3 + 4 + \left(\frac{9}{2} i - \frac{1}{2} \right) + 3 \right) \right) + 3 = \frac{9}{2} n^2 + \frac{47}{4} n - 11.$$

Apunte de Cátedra

Por el modo en que funciona el algoritmo, tales casos van a corresponder a cuando el arreglo se encuentra ordenado de forma creciente, decreciente o aleatoria.

Como podemos ver, en este método los órdenes de complejidad de los casos peor, mejor y medio difieren bastante. Así, en el mejor caso el orden de complejidad resulta ser lineal, mientras que en los casos peor y medio su complejidad es cuadrática.

Este método se muestra muy adecuado para aquellas situaciones en donde necesitamos ordenar un arreglo del que ya conocemos que está casi ordenado, como suele suceder en aquellas aplicaciones de inserción de elementos en bancos de datos previamente ordenados cuya ordenación total se realiza periódicamente.

2.2.- Ordenamiento por Selección

En cada paso ($i=1\dots n-1$) este método busca el mínimo elemento del arreglo $vec[i\dots n]$ y lo intercambia con el elemento en la posición i :

```
public static void selectionSort(int[] vec)
{
    for(int i=0; i<vec.length - 1; i++)
    {
        int pos=i;
        for(int j=i + 1; j<vec.length; j++)
            if(vec[j]<vec[pos])
                pos=j;
        interChange(vec, pos, i);
    }
}
```

En cuanto a su complejidad, vamos a estudiar los casos mejor, peor y medio de la llamada al método *selectionSort(vec)*.

- En el mejor caso:

$$T(n) = \left(\sum_{i=0}^{n-2} (3 + 1 + (5 + 6(n - i)) + 1 + 7) \right) + 3 = 3n^2 + 14n - 14.$$

- En el peor caso:

$$T(n) = \left(\sum_{i=0}^{n-2} (3 + 1 + (5 + 7(n - i)) + 1 + 7) \right) + 3 = \frac{7}{2} n^2 + \frac{27}{2} n - 14.$$

- En el caso medio:

$$T(n) = \left(\sum_{i=0}^{n-2} \left(3 + 1 + \left(5 + \frac{13}{2} (n - i) \right) + 1 + 7 \right) \right) + 3 = \frac{13}{4} n^2 + \frac{55}{4} n - 14.$$

En consecuencia, el algoritmo es de complejidad cuadrática.

Este método, por el número de operaciones de comparación e intercambio que realiza, es el más adecuado para ordenar pocos registros de gran tamaño. Si el tipo base del arreglo a ordenar no es entero, sino un tipo

Apunte de Cátedra

más complejo (guías telefónicas, índices de libros, historiales hospitalarios, etc.) deberemos darle mayor importancia al intercambio de valores que a la comparación entre ellos en la valoración del algoritmo por el coste que suponen. En este sentido, analizando el número de intercambios que realiza el método de Selección vemos que es de orden $O(n)$, frente al orden $O(n^2)$ de intercambios que presentan los métodos de Inserción o Intercambio (Burbuja).

2.3.- Ordenamiento por Intercambio (Burbuja)

Este método de ordenación consiste en recorrer los elementos siempre en la misma dirección, intercambiando elementos adyacentes si fuera necesario:

```
public static void bubbleSort(int[] vec)
{
    for(int i=vec.length - 1; i>=1; i--)
        for(int j=0; j<=i - 1; j++)
            if(vec[j]>vec[j + 1])
                interChange(vec, j, j + 1);
}
```

El nombre de este algoritmo trata de reflejar cómo el elemento mínimo “sube”, a modo de burbuja, hasta el principio del arreglo.

Respecto a su complejidad, vamos a estudiar los casos mejor, peor y medio de la llamada al método *bubbleSort(vec)*.

- En el mejor caso:

$$T(n) = \left(\sum_{i=1}^{n-1} \left(3 + \sum_{j=0}^{i-1} (3 + 4) + 3 \right) \right) + 3 = \frac{7}{2} n^2 + \frac{5}{2} n - 3.$$

- En el peor caso:

$$T(n) = \left(\sum_{i=1}^{n-1} \left(3 + \sum_{j=0}^{i-1} (3 + 4 + 2 + 7) + 3 \right) \right) + 3 = 8n^2 - 2n - 1.$$

- En el caso medio:

$$T(n) = \left(\sum_{i=1}^{n-1} \left(3 + \sum_{j=0}^{i-1} \left(3 + 4 + \frac{2+7}{2} \right) + 3 \right) \right) + 3 = \frac{23}{4} n^2 + \frac{1}{4} n - 1.$$

En consecuencia, el algoritmo es de complejidad cuadrática.

Este algoritmo funciona de forma parecida al de Selección, pero haciendo más trabajo para llevar cada elemento a su posición. De hecho es el peor de los tres vistos hasta ahora, no sólo en cuanto al tiempo de ejecución, sino también respecto al número de comparaciones y de intercambios que realiza.

Una posible mejora que puede admitir este algoritmo es el control de la existencia de una pasada sin intercambios; en ese momento el arreglo estará ordenado.

Apunte de Cátedra

2.4.- Ordenamiento por Intercambio (Burbuja Mejorado)

```
public static void bubbleImprovedSort(int[] vec)
{
    int i=vec.length - 1;
    boolean ordered;
    do
    {
        ordered=true;
        for(int j=0; j<=i - 1; j++)
            if(vec[j]>vec[j+1])
            {
                interChange(vec, j, j + 1);
                ordered=false;
            }
        i--;
    }
    while(i>=1 & !ordered);
}
```

Dejamos el análisis del algoritmo para que se investigue.

3.- ALGORITMOS AVANZADOS DE ORDENAMIENTO

3.1.- Ordenamiento por Mezcla (MERGESORT)

Este método utiliza la técnica de “Divide y Vencerás” para realizar la ordenación del arreglo. Su estrategia consiste en dividir el arreglo en dos arreglos, ordenarlos mediante llamadas recursivas, y finalmente combinar los dos arreglos ya ordenados. Esta idea da lugar a la siguiente implementación:

```
public static void mergeSort(int[] vec)
{
    int[] tmp=new int[vec.length];
    mergeSort(vec, tmp, 0, vec.length-1);
}

private static void mergeSort(int[] vec, int[] tmp, int left, int right)
{
    if(left<right)
    {
        int center=(right + left) / 2;
        mergeSort(vec, tmp, left, center);
        mergeSort(vec, tmp, center+1, right);
        merge(vec, tmp, left, center, right);
    }
}

private static void merge(int[] vec, int[] tmp, int left, int center, int right)
{
    int aptr=left;
    int bptr=center+1;
    int cptr=left;

    while(aptr<=center & bptr<=right)
        if(vec[aptr]<vec[bptr])
            tmp[cptr++]=vec[aptr++];
        else
            tmp[cptr++]=vec[bptr++];

    while(aptr<=center)
        tmp[cptr++]=vec[aptr++];

    while(bptr<=right)
        tmp[cptr++]=vec[bptr++];

    for(int i=left; i<=right; i++)
        vec[i]=tmp[i];
}
```

Apunte de Cátedra

Una posible implementación del método que lleva a cabo el proceso de mezcla vuelca primero los elementos a ordenar en el arreglo auxiliar para después, utilizando dos índices, uno para cada arreglo, rellenar el arreglo ordenadamente. Nótese que el algoritmo *merge* utiliza el hecho de que los dos arreglos están ya ordenados y que son además consecutivos.

En cuanto al estudio de su complejidad, siguiendo la metodología aplicada anteriormente, se llega a que el tiempo de ejecución de *mergeSort(vec)* puede expresarse mediante una ecuación en recurrencia:

$$T(n) = 2T(n/2) + 16n + 17$$

con la condición inicial $T(1) = 1$. Ésta es una ecuación en recurrencia no homogénea cuya ecuación característica asociada es $(x-2)^2(x-1) = 0$, lo que permite expresar $T(n)$ como:

$$T(n) = c_1n + c_2n\log n + c_3.$$

El cálculo de las constantes puede hacerse en base a la condición inicial, lo que nos lleva a la expresión final:

$$T(n) = 16n\log n + 18n - 17 \in \Theta(n\log n).$$

Obsérvese que este método ordena n elementos en tiempo $\Theta(n\log n)$ en cualquiera de los casos (peor, mejor o medio). Sin embargo tiene una complejidad espacial, en cuanto a memoria, mayor que los demás (del orden de n).

Otras versiones de este algoritmo no utilizan el arreglo auxiliar, sino que trabajan sobre el propio arreglo a ordenar, combinando sobre él los arreglos obtenidos de las etapas anteriores. Si bien es cierto que esto consigue ahorrar espacio (un arreglo auxiliar), también complica el código del algoritmo resultante.

El método de ordenación por Mezcla se adapta muy bien a distintas circunstancias, por lo que es comúnmente utilizado no sólo para la ordenación de arreglos. Por ejemplo, el método puede ser también implementado de forma que el acceso a los datos se realice de forma secuencial, por lo que hay diversas estructuras (como las listas enlazadas) para las que es especialmente apropiado. También se utiliza para realizar ordenación externa, en donde el arreglo a ordenar reside en dispositivos externos de acceso secuencial (ficheros).

3.2.- Ordenamiento mediante Montículos (HEAPSORT)

La filosofía de este método de ordenación consiste en aprovechar la estructura particular de los montículos (heaps), que son árboles binarios completos (todos sus niveles están llenos salvo a lo sumo el último, que se rellena de izquierda a derecha) y cuyos nodos verifican la propiedad del montículo: todo nodo es mayor o igual que cualquiera de sus hijos. En consecuencia, en la raíz se encuentra siempre el elemento mayor.

Estas estructuras admiten una representación muy sencilla, compacta y eficiente mediante arreglos (por ser árboles completos). Así, en un arreglo que represente una implementación de un montículo se cumple que el “padre” del i -ésimo elemento del arreglo se encuentra en la posición $i \div 2$ (menos la raíz, claro) y sus “hijos”, si es que los tiene, estarán en las posiciones $2i$ y $2i+1$ respectivamente.

La idea es construir, con los elementos a ordenar, un montículo sobre el propio arreglo. Una vez construido el montículo, su elemento mayor se encuentra en la primera posición del arreglo. Se intercambia entonces con el último y se repite el proceso para el arreglo con la primera posición $+ 1$ y la última posición $- 1$. Así sucesivamente hasta recorrer el arreglo completo. Esto nos lleva a un algoritmo de orden de complejidad $O(n\log n)$ cuya implementación puede ser la siguiente:

```
public static void heapSort(int[] vec)
{
    int index;

    for(index=vec.length - 1; index>=0; index--)
```

Apunte de Cátedra

```

        reHeapDown(vec, index, vec.length);
    for(index=vec.length - 1; index>0; index--)
    {
        interChange(vec, 0, index);
        reHeapDown(vec, 0, index);
    }
}

private static void reHeapDown(int vec[], int length, int index)
{
    boolean done=false;
    int aux=vec[length];
    int parent=length;
    int child=2 * (length + 1) - 1;

    while(child<index & !done)
    {
        if(child<index - 1)
            if(vec[child]<vec[child + 1])
                child++;

        if(aux>=vec[child])
            done=true;
        else
        {
            vec[parent]=vec[child];
            parent=child;
            child=2 * (parent + 1) - 1;
        }
    }
    vec[parent]=aux;
}

```

Los métodos *heapSort* y *reHeapDown* son, respectivamente, el que construye un montículo a partir del arreglo dado, y el que “empuja” un elemento hasta su posición definitiva en el montículo, reconstruyendo la estructura de montículo en el arreglo.

Para estudiar la complejidad del algoritmo hemos de considerar dos partes. La primera es la que construye inicialmente el montículo a partir de los elementos a ordenar y la segunda va recorriendo en cada iteración un arreglo más pequeño, colocando el elemento raíz en su posición correcta dentro del montículo. En ambos casos nos basamos en el método que “empuja” elementos en el montículo. Observando el comportamiento del algoritmo, la diferencia básica entre el caso peor y el mejor está en la profundidad que hay que recorrer cada vez que necesitamos “empujar” un elemento. Podríamos llegar a tener que recorrer todo el árbol (profundidad: $\log n$) dependiendo del elemento a “empujar”, respecto al resto.

El método *heapSort* es de complejidad $O(n)$ en el peor caso, puesto que si k es la altura del montículo ($k = \log n$), el algoritmo transforma primero cada uno de los dos subárboles que cuelgan de la raíz en montículos de altura a lo sumo $k-1$ (el subárbol derecho puede tener altura $k-2$), y después empuja la raíz hacia abajo, por un camino que a lo sumo es de longitud k . Esto lleva a lo sumo un tiempo $t(k)$ de orden de complejidad $O(k)$ con lo cual

$$T(k) = 2T(k-1) + t(k),$$

ecuación en recurrencia cuya solución verifica que $T(k) \in (2^k)$. Como $k = \log n$, la complejidad de *heapSort* es lineal en el peor caso. Este caso ocurre cuando hay que recorrer siempre la máxima profundidad al empujar a cada elemento, lo que sucede si el arreglo está originalmente ordenado de forma creciente.

Respecto al mejor caso de *heapSort*, éste se presenta cuando la profundidad a la que hay que empujar cada elemento es cero. Esto se da, por ejemplo, si todos los elementos del arreglo son iguales. En esta situación la complejidad del algoritmo es $O(1)$.

Estudiemos ahora los casos mejor y peor del resto del algoritmo *heapSort*. En esta parte hay un bucle que se ejecuta siempre $n-1$ veces, y la complejidad del método que intercambia dos elementos es $O(1)$. Todo va a depender del método *reHeapDown*, es decir, de la profundidad a la que haya que empujar la raíz del

Apunte de Cátedra

montículo en cada iteración, sabiendo que cada montículo tiene $n-i$ elementos, y por lo tanto, una altura de $\log(n-i)$, siendo i el número de la iteración.

En el peor caso, la profundidad a la que hay que empujar las raíces respectivas es la máxima, y por lo tanto, la complejidad de esta segunda parte del algoritmo es $O(n \log n)$. ¿Cuándo ocurre esto?... cuando el elemento es menor que todos los demás. Pero esto sucede siempre que los elementos a ordenar sean distintos, por la forma en la que se van escogiendo las nuevas raíces.

En el mejor caso, aunque el bucle se sigue repitiendo $n-1$ veces, las raíces no descienden, por ser mayores o iguales que el resto de los elementos del montículo. Así, la complejidad de esta parte del algoritmo es de orden $O(n)$. Pero este caso sólo se dará si los elementos del arreglo son iguales, por la forma en la que originariamente se construyó el montículo y por cómo se escoge la nueva raíz en cada iteración (el último de los elementos, que en un montículo ha de ser de los menores).

3.3.- Ordenamiento Rápida de Hoare (QUICKSORT)

Este método es probablemente el algoritmo de ordenación más utilizado, pues es muy fácil de implementar, trabaja bien en casi todas las situaciones y consume en general menos recursos (memoria y tiempo) que otros métodos.

Su diseño está basado en la técnica de “Divide y Vencerás”, que estudiaremos a continuación, y consta de dos partes:

- En primer lugar el arreglo a ordenar es dividido en dos arreglos no vacíos, tal que todos los elementos del primero son menores que los del segundo. Forma parte de esto, un pivote (elemento dentro del conjunto) como protagonista esencial en el método de partición.
- A continuación, los dos arreglos son ordenados mediante llamadas recursivas al método quickSort. Como los arreglos se ordenan sobre ellos mismos, no es necesario realizar ninguna operación de combinación.

El siguiente método constituye la versión clásica del algoritmo de ordenación rápida de Hoare:

```
public static void quickSort(int[] vec)
{
    quickSort(vec, 0, vec.length-1);
}

private static void quickSort(int[] vec, int first, int last)
{
    int center=(first + last) / 2;
    if(first<last)
    {
        center=division(vec, first, last, center);
        quickSort(vec, first, center);
        if(center!=first)
            quickSort(vec, center + 1, last);
        else
            quickSort(vec, center, last);
    }
}

private static int division(int[] vec, int first, int last, int center)
{
    int left, right, data;
    data=vec[center];
    left=first;
    right=last;
    do
    {
        while(left<right && vec[left]<data)
            left++;
        while(left<right && vec[right]>data)
            right--;

        if(left<right)
```

Apunte de Cátedra

```

        {
            interChange(vec, left, right);
            left++;
            right--;
        }
    }
    while(left < right);
    if(left < right)
    {
        int pos = right;
        right = left;
        left = pos;
    }

    return left;
}

```

El método *division* parte del elemento pivote y permuta los elementos del arreglo de forma que al finalizar el método, todos los elementos menores o iguales que el pivote estén a su izquierda, y los elementos mayores que él a su derecha. Devuelve la posición en la que ha quedado situado el pivote.

Este método es de orden de complejidad $\Theta(n^2)$ en el peor caso y $\Theta(n \log n)$ en los casos mejor y medio. Obtenemos la siguiente ecuación en recurrencia:

$$T(n) = 8 + T(a) + T(b) + T_{\text{Pivote}}(n)$$

donde a y b son los tamaños en los que el método *division* divide al arreglo (por lo tanto, podemos tomar que $a + b = n$), y $T_{\text{Pivote}}(n)$ es el método que define el tiempo de ejecución del método *division*.

El método *quickSort* “rompe” la filosofía de mejor caso, peor y medio de los algoritmos clásicos de ordenación, pues aquí tales casos no dependen de la ordenación inicial del arreglo, sino de la elección del pivote.

Así, el mejor caso ocurre cuando $a = b = n/2$ en todas las invocaciones recursivas del método, pues en este caso obtenemos $T_{\text{Pivote}}(n) = 13 + 4n$, y por lo tanto:

$$T(n) = 21 + 4n + 2T(n/2).$$

Resolviendo esta ecuación en recurrencia y teniendo en cuenta las condiciones iniciales $T(0) = 1$ y $T(1) = 27$ se obtiene la expresión final de $T(n)$, en este caso:

$$T(n) = 15n \log n + 26n + 1.$$

Ahora bien, si $a = 0$ y $b = n-1$ (o viceversa) en todas las invocaciones recursivas del método, $T_{\text{Pivote}}(n) = 11 + 39/8n$, obteniendo:

$$T(n) = 19 + 39/8n + T(n-1).$$

Resolviendo la ecuación para las mismas condiciones iniciales, nos encontramos con una desagradable sorpresa:

$$T(n) = \frac{3}{8} n^2 + \frac{213}{8} n + 1 \in \Theta(n^2).$$

En consecuencia, la elección idónea para el pivote es la mediana del arreglo en cada etapa, lo que ocurre es que encontrarla requiere un tiempo extra que hace que el algoritmo se vuelva más ineficiente en la mayoría de los casos. Por esa razón como pivote suele escogerse un elemento cualquiera, a menos que se conozca la naturaleza de los elementos a ordenar. En nuestro caso, como a priori suponemos equiprobable cualquier ordenación inicial del arreglo, hemos escogido el elemento central del arreglo.

Apunte de Cátedra

Esta elección lleva a tres casos desfavorables para el algoritmo: cuando los elementos son todos iguales y cuando el arreglo está inicialmente ordenado en orden creciente o decreciente. En estos casos la complejidad es cuadrática puesto que la partición se realiza de forma totalmente descompensada.

A pesar de ello suele ser el algoritmo más utilizado, y se demuestra que su tiempo promedio es menor, en una cantidad constante, al de todos los algoritmos de ordenación de complejidad $O(n \log n)$. En todo esto es importante hacer notar, como hemos indicado antes, la relevancia que toma una buena elección del pivote, pues de su elección depende considerablemente el tiempo de ejecución del algoritmo.

Sobre el algoritmo expuesto anteriormente pueden realizarse varias mejoras:

1. Respecto a la elección del pivote. En vez de tomar como pivote el elemento central, puede seguirse alguna estrategia del tipo:

- Tomar al azar tres elementos seguidos del arreglo y escoger como pivote el elemento medio de los tres.
- Tomar k elementos al azar, clasificarlos por cualquier método, y elegir el elemento medio como pivote.

2. Con respecto al tamaño de los arreglos a ordenar. Cuando el tamaño de éstos sea pequeño (menor que una cota dada), es posible utilizar otro algoritmo de ordenación en vez de invocar recursivamente a *quickSort*. Esta idea utiliza el hecho de que algunos métodos, como Selección o Inserción, se comportan muy bien cuando el número de datos a ordenar son pocos, por disponer de constantes multiplicativas pequeñas. Aun siendo de orden de complejidad cuadrática, son más eficientes que los de complejidad $n \log n$ para valores pequeños de n .

3.4.- Ordenamiento por Incrementos (SHELLSORT)

Conocida como disminución incremental y nombrada así debido a su inventor Donald Shell.

La ordenación por intercambio puede resultar lenta pues sólo intercambia elementos adyacentes. Así, si por ejemplo el elemento menor está al final del arreglo, hacen falta n pasos para colocarlo donde corresponde. El método de incrementos es una extensión muy simple y eficiente del método de burbuja en el que cada elemento se coloca casi en su posición definitiva en la primera pasada.

El algoritmo consiste básicamente en dividir el arreglo en h arreglos:

$$\text{vec}[k], \text{vec}[k + h], \text{vec}[k + 2h], \text{vec}[k + 3h], \dots$$

y ordenar cada uno de esos arreglos ($k=1,2,\dots,h-1$).

Un arreglo de esta forma, es decir, compuesto por h arreglos ordenados intercalados, se denomina h -ordenado. Haciendo h -ordenaciones de vec para valores grandes de h permitimos que los elementos puedan moverse grandes distancias dentro del arreglo, facilitando así las h -ordenaciones para valores más pequeños de h . A h se le denomina incremento.

Con esto, el método de ordenación por incrementos consiste en hacer h -ordenaciones de vec para valores de h decreciendo hasta llegar a uno.

El número de comparaciones que se realizan en este algoritmo va a depender de la secuencia de incrementos h , y será mayor que en el método clásico de burbuja (que se ejecuta finalmente para $h = 1$), pero la potencia de este método consiste en conseguir un número de intercambios mucho menor que con la inserción clásica.

El método presentado a continuación utiliza la secuencia de incrementos $h = \dots, 1093, 364, 121, 40, 13, 1$. Otras secuencias pueden ser utilizadas, pero la elección ha de hacerse con cuidado. Por ejemplo la secuencia $\dots, 64, 32, 16, 8, 4, 2, 1$ es muy ineficiente pues los elementos en posiciones pares e impares no son comparados hasta el último momento.

```
public static void shellSort(int[] vec)
{
    int interval=vec.length / 2;
    while(interval>0)
    {
        for(int i=interval; i<vec.length; i++)
```

Apunte de Cátedra

```

    {
        int j=i - interval;
        while(j>=0)
        {
            if(vec[j]<=vec[j + interval])
                j=0;
            else
                interChange(vec, j, j + interval);
            j-=interval;
        }
        interval/=2;
    }
}

```

En cuanto al estudio de su complejidad, este método es diferente al resto de los métodos vistos en este apunte. Su complejidad es difícil de calcular y depende mucho de la secuencia de incrementos que utilice. Por ejemplo, para la secuencia dada existen dos conjeturas en cuanto a su orden de complejidad: $n \log^2 n$ y $n^{1.25}$. En general este método es el escogido para muchas aplicaciones reales por ser muy simple teniendo un tiempo de ejecución aceptable incluso para grandes valores de n .

3.5.- Ordenamiento por Sacudida o Vibración (SHAKERSORT)

La idea básica de este algoritmo consiste en mezclar las dos formas en que se puede realizar el método de la burbuja. En este algoritmo cada pasada tiene dos etapas. En la primera etapa “de derecha a izquierda” se trasladan los elementos más pequeños hacia la parte izquierda del arreglo, almacenando en una variable la posición del último elemento intercambiado. En la segunda etapa “de izquierda a derecha” se trasladan los elementos más grandes hacia la parte derecha del arreglo, almacenando en otra variable la posición del último elemento intercambiado. Las sucesivas pasadas trabajan con los componentes del arreglo comprendidos entre las posiciones almacenadas en las variables. El algoritmo termina cuando en una etapa no se producen intercambios, o bien cuando el contenido de la variable que almacena el extremo izquierdo del arreglo es mayor que el contenido de la variable que almacena el extremo derecho.

Este algoritmo tiene la ventaja que reduce considerablemente el número de comparaciones cuando los elementos están casi ordenados (se va directamente al elemento que falta ordenar).

```

public static void shakerSort(int[] vec)
{
    int left=1;
    int right=vec.length - 1;
    int aux=vec.length - 1;

    do
    {
        for(int i=right; i>=left; i--)
            if(vec[i - 1]>vec[i])
            {
                interChange(vec, i - 1, i);
                aux=i;
            }

        left=aux + 1;
        for(int i=left; i<=right; i++)
            if(vec[i - 1]>vec[i])
            {
                interChange(vec, i - 1, i);
                aux=i;
            }

        right=aux - 1;
    }
    while(left<right);
}

```

El análisis del método por sacudida y en general el de los métodos mejorados y logarítmicos es muy complejo. Para el análisis de este método es necesario tener en cuenta tres factores que afectan directamente al tiempo de ejecución del algoritmo: las comparaciones entre los elementos, los intercambios entre los

Apunte de Cátedra

mismos y las pasadas que se realizan. Encontrar fórmulas que permitan calcular cada uno de estos factores es una tarea muy difícil de realizar.

Los estudios que se han efectuado sobre el método por sacudida demuestran que en el mismo, sólo pueden reducirse las dobles comparaciones entre elementos; pero debe recordarse que la operación de intercambio es una tarea más complicada y costosa que la de comparación. Por lo tanto, es posible afirmar que las hábiles mejoras realizadas sobre el método de intercambio sólo producen resultados apreciables si el arreglo está parcialmente ordenado (lo cual resulta difícil saber de antemano), pero si el arreglo está desordenado el método se comporta, incluso, peor que otros métodos directos como los de inserción y selección.

Algo muy interesante para tener en cuenta es que recientemente Brona Brejova (especialista en Ciencias de la Computación de Canadá) probó que shakersort tiene un orden de complejidad $O(n \log n)$ en el peor caso para ciertas secuencias de incrementos (el primer salto más alto no cuadrático del peor caso). Usando un método de incomprensibilidad, también probó el salto más chico sobre tiempos de ejecución en el caso medio.

4.- OTROS ALGORITMOS AVANZADOS DE ORDENAMIENTO

Los algoritmos vistos hasta ahora se basan en la ordenación de arreglos de números enteros, sin ningún tipo de restricción. A continuación veremos cómo pueden encontrarse algoritmos de orden $O(n)$ cuando dispongamos de información adicional sobre los valores a ordenar. Además, como lograr ordenar sin la necesidad de comparaciones e intercambios.

4.1.- Ordenamiento por Urnas (BINSORT)

Suponemos que los datos a ordenar son números naturales, todos distintos y comprendidos en el intervalo $[1, n]$. Es decir, nuestro problema es ordenar un arreglo con los n primeros números naturales. Bajo esas circunstancias es posible implementar un algoritmo de complejidad temporal $O(n)$. Es el método de ordenación por Urnas, en donde en cada iteración se sitúa un elemento en su posición definitiva:

```
public static void binSort(int[] vec)
{
    //crea la urna
    Queue[] urn=new Queue["elemento más grande en el arreglo más 1*"];
    for(int i=0; i<urn.length; i++)
        urn[i]=new Queue();

    //guarda cada elemento del arreglo en su correspondiente urna
    for(int i=0; i<vec.length; i++)
        urn[getKey(vec[i])].insert(new Integer(vec[i]));

    //recupera cada elemento de las distintas urnas y lo guarda en el arreglo
    int j=0;
    for(int i=0; i<urn.length; i++)
        while(!urn[i].isEmpty())
        {
            vec[j]=((Integer)urn[i].delete()).intValue();
            j++;
        }
}
```

4.2.- Ordenamiento por Residuos (RADIXSORT)

Este método puede utilizarse cuando los valores a ordenar están compuestos por secuencias de letras o dígitos que admiten un orden lexicográfico. Éste es el caso de palabras, números (cuyos dígitos admiten este orden) o bien fechas.

El método consiste en definir k colas (numeradas de 0 a $k-1$) siendo k los posibles valores que puede tomar cada uno de los dígitos que componen la secuencia. Una vez que tengamos las colas habrá que repetir, para i a partir de 0 y hasta llegar al número máximo de dígitos o letras de nuestras cadenas:

1. Distribuir los elementos en las colas en función del dígito i .
2. Extraer ordenada y consecutivamente los elementos de las colas, introduciéndolos de nuevo en el arreglo.

Apunte de Cátedra

Veamos un ejemplo de este método. Supongamos el arreglo:

[0, 1, 81, 64, 23, 27, 4, 25, 36, 16, 9, 49].

En este caso se trata de números naturales en base 10, que no son sino secuencias de dígitos. Como cada uno de los dígitos puede tomar 10 valores (del 0 al 9), necesitaremos 10 colas. En la primera pasada introducimos los elementos en las colas de acuerdo a su dígito menos significativo:

	0	81			4		16			49
	0	1		23	64	25	36	27		9
Colas	0	1	2	3	4	5	6	7	8	9

y ahora extraemos ordenada y sucesivamente los valores, obteniendo el arreglo:

[0, 1, 81, 23, 64, 4, 25, 36, 16, 27, 9, 49].

Volvemos a realizar otra pasada, esta vez fijándonos en el segundo dígito menos significativo:

	9									
	4		27							
	1		25							
	0	16	23	36	49		64		81	
Colas	0	1	2	3	4	5	6	7	8	9

Volviendo a extraer ordenada y sucesivamente los valores obtenemos el arreglo:

[0, 1, 4, 9, 16, 23, 25, 27, 36, 49, 64, 81].

Como el máximo de dígitos de los números a ordenar era de dos, con dos pasadas hemos tenido suficiente. La implementación de este método es el siguiente:

```
//Parte del código de este algoritmo es similar al de binSort.
//Por lo tanto, sería óptimo que se reutilice código.
public static void radixSort(int vec[])
{
    for(int d=1; d<=/*cantidad de dígitos del máximo elemento en el arreglo*/; d++)
    {
        //crea la urna
        Queue[] urn=new Queue[10];
        for(int i=0; i<urn.length; i++)
            urn[i]=new Queue();

        //guarda cada elemento del arreglo en su correspondiente urna
        for(int i=0; i<vec.length; i++)
            urn[getKey(vec[i], d)].insert(new Integer(vec[i]));

        //recupera cada elemento de las distintas urnas y lo guarda en el arreglo
        int j=0;
        for(int i=0; i<urn.length; i++)
            while(!urn[i].isEmpty())
            {
                vec[j]=((Integer)urn[i].delete()).intValue();
                j++;
            }
    }
}
```